Clase 07: Paralelismo (continuación)

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Jueves 26 de Marzo del 2026

Métricas

Paralelismo

Computador paralelo: posee más de una unidad de cómputo para trabajar en el mismo problema.

- Si hay un solo núcleo, ILP (Instruction Level Parallelism)
- ILP: determinado por arquitectura y compilador.
- En general: más de un flujo de instrucciones manejado por distintos núcleos.
- Esto se puede programar explícitamente.

Paralelismo

En muchas ocasiones, el trabajo corresponde a la misma operación aplicada a muchos datos.

¿Cuánto puede ser procesado al mismo tiempo por distntas unidades de cómputo?

- N operaciones a ejecutar.
- Un procesador toma tiempo T.
- Si hay p procesadores, ¿pueden las N operaciones ser procesadas en tiempo $\frac{T}{p}$?

Tipos de Paralelismo

Data parallelism

- La misma operación aplicada en paralelo a muchos datos.
- Común en vectores, matrices, grafos.

Functional parallelism

- Instrucciones distintas dentro de un algoritmo se envían a unidades de cómputo distintas.
- ILP es un caso particular. *Pipelining*.

Task parallelism

- Trabajos independientes se pueden realizar de manera simultánea.
- Paradigma de *multiprocessing*.

Speedup

Compara el tiempo con un procesador, versus el tiempo con múltiples procesadores.

- T_1 : Tiempo de ejecución con 1 unidad de cómputo.
- T_p : Tiempo de ejecución con p unidades de cómputo.

El **Speedup** es

$$S_p = \frac{T_1}{T_p}$$

Eficiencia

Al evaluar los tiempos de ejecución en paralelo

- Caso ideal: tiempo con p procesadores es $T_p = \frac{T_1}{p}$, es decir $S_p = p$.
- En la práctica tenemos $S_p \leq p$.

Llamamos **Eficiencia** a la métrica

$$E_p = \frac{S_p}{p}.$$

Si $S_p \leq p$, entonces $0 < E_p \leq 1$.

Costo

Si el *speedup* no es perfecto, se puede definir el *overhead* como

$$T_0 = pT_p - T_1$$

Puede ocurrir

- El algoritmo paralelo puede no ser el mismo que el algoritmo secuencial.
- El algoritmo paralelo puede requerir enviar mensajes de coordinación previo al cómputo.

El algoritmo es *cost-optimal* si el *overhead* es del orden de ejecución del algoritmo secuencial.

Escalabilidad

Un algoritmo paralelo escala bien si al aumentar los recursos de cómputo, el rendimiento mejora de manera predecible y sostenida.

- No basta con que el algoritmo sea rápido en pocas CPUs: queremos saber cómo se comporta al crecer.
- Hay dos regímenes de crecimiento posibles:
  - **Escalabilidad fuerte:** más procesadores para el mismo problema.
  - **Escalabilidad débil:** más procesadores para un problema más grande.
- Ambos tipos se miden con una eficiencia cuyo ideal es 1.

Escalabilidad fuerte

Problema fijo, se aumenta el número de procesadores p.

- Objetivo: reducir el tiempo de ejecución al agregar más procesadores.
- Se mide con la eficiencia (fuerte):

$$E_p = \frac{S_p}{p} = \frac{T_1}{p\,T_p} \in (0, 1].$$

- Caso ideal: $T_p = T_1/p \Rightarrow E_p = 1$.
- Experimento: fijar N, variar p, reportar E_p.
- Limitada por la ley de Amdahl: la fracción serial acota el speedup.

Tamaño de un problema

El tamaño de un problema es la cantidad de trabajo que realiza el algoritmo serial. Depende del problema:

Reducción (mínimo, suma, ...)	N elementos
Multiplicación de matrices	$N \times N$ elementos
Procesamiento de texto	N caracteres

En escalabilidad débil, al usar p procesadores el problema tiene tamaño $N = p \cdot M$, donde M es la carga por procesador.

Escalabilidad débil

Problema escala con p : cada procesador mantiene la misma carga $M = N/p$.

- Objetivo: resolver problemas más grandes sin aumentar el tiempo.
- Se mide con la eficiencia débil:

$$E_p^w = \frac{T_1}{T_p} \in (0, 1].$$

  donde T_1 es el tiempo del problema base en 1 procesador.
- Caso ideal: $T_p = T_1$ (mismo trabajo por procesador) $\Rightarrow E_p^w = 1$.
- Experimento: fijar $M = N/p$, variar p, reportar E_p^w.

Calcular el mínimo

Considere el siguiente código secuencial que calcula el valor mínimo de un arreglo:

```
1        double min_val = x[0];
2
3        for (int i = 1; i < N; i++) {
4           if (x[i] < min_val){
5              min_val = x[i];
6           }
7        }
```

Se desea paralelizar esta operación usando p procesadores.

Calcular el mínimo

Considere el siguiente código secuencial que calcula el valor mínimo de un arreglo:

```
1        double min_val = x[0];
2
3        for (int i = 1; i < N; i++) {
4           if (x[i] < min_val){
5              min_val = x[i];
6           }
7        }
```

Se desea paralelizar esta operación usando p procesadores.

Existen varias opciones de cómo paralelizar. ¿Qué las diferencia?

Calcular el mínimo

Estrategia #1

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados locales de manera **secuencial**.

¿Cuáles son las métricas en este caso?

Calcular el mínimo

Estrategia #1

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados locales de manera **secuencial**.

El tiempo con un procesador es $T_1 = N$,

y con p procesadores es $T_p = \dfrac{N}{p} + p$.

Calcular el mínimo

Estrategia #1

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados locales de manera **secuencial**.

El *Speedup* es $S_p = \frac{T_1}{T_p} = \frac{N}{\frac{N}{p} + p} = \frac{1}{\frac{1}{p} + \frac{p}{N}}$

Calcular el mínimo

Estrategia #1

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados locales de manera **secuencial**.

El *Speedup* es $S_p = \frac{T_1}{T_p} = \frac{N}{\frac{N}{p} + p} = \frac{1}{\frac{1}{p} + \frac{p}{N}}$

Modelo de Amdahl: ¿Qué es lo que falló?

Calcular el mínimo

Estrategia #1

- ¿Por qué pareciera que no se está cumpliendo la ley de Amdahl?
- El *Speedup* es

$$S_p = \frac{T_1}{T_p} = \frac{N}{\frac{N}{p} + p} = \frac{1}{\frac{1}{p} + \frac{p}{N}} \to 0$$

  cuando $p \to \infty$.
- La ley de Amdahl dice que $S_p \to \frac{1}{f_s} < \infty$.

Calcular el mínimo

Estrategia #1

- ¿Por qué pareciera que no se está cumpliendo la ley de Amdahl?
- El *Speedup* es

$$S_p = \frac{T_1}{T_p} = \frac{N}{\frac{N}{p} + p} = \frac{1}{\frac{1}{p} + \frac{p}{N}} \to 0$$

  cuando $p \to \infty$.
- La ley de Amdahl dice que $S_p \to \frac{1}{f_s} < \infty$.
- Notar que la fracción secuencial y paralela cambian con el número de procesadores!
- La fracción secuencial no es constante, de hecho corresponde a la suma de los resultados locales.
- Lo correcto es decir que la parte secuencial corresponde a $\text{mín}(N, p)$ operaciones.

Calcular el mínimo

Estrategia #1

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados locales de manera **secuencial**.

Modificar el código introdujo un *overhead* :

$$T_0 = pT_p - T1 = p\left(\frac{N}{p} + p\right) - N = p^2$$

¿Cuándo es esto comparable al costo secuencial? – *cost optimal*

Calcular el mínimo

Estrategia #1

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados locales de manera **secuencial**.

El *Speedup* es $S_p = \dfrac{T_1}{T_p} = \dfrac{N}{\frac{N}{p} + p} = \dfrac{1}{\frac{1}{p} + \frac{p}{N}}$

La eficiencia es $E_p = \dfrac{S_p}{p} = \dfrac{1}{1 + \frac{p^2}{N}}$: si $p \sim \sqrt{N}$, entonces $E_p \sim \mathrm{const}$

Calcular el mínimo

Estrategia #1

Escalabilidad fuerte (N fijo):

$$E_p = \frac{1}{1 + \frac{p^2}{N}} \xrightarrow{p \to \infty} 0.$$

La eficiencia cae rápido: el overhead secuencial p domina.

Escalabilidad débil ($M = N/p$ fijo, $N = pM$):

$$T_p = M + p, \qquad E_p^w = \frac{T_1}{T_p} = \frac{M}{M + p} \xrightarrow{p \to \infty} 0.$$

Tampoco escala débilmente: el overhead crece linealmente con p.

Calcular el mínimo

Estrategia #2

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados en paralelo con un árbol binario.

¿Cuáles son las métricas en este caso?

Calcular el mínimo

Estrategia #2

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados en paralelo con un árbol binario.

El tiempo con un procesador es $T_1 = N$,

y con p procesadores es $T_p = \dfrac{N}{p} + \log_2 p$.

Calcular el mínimo

Estrategia #2

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados en paralelo con un árbol binario.

El *Speedup* es $S_p = \frac{T_1}{T_p} = \frac{N}{\frac{N}{p} + \log_2 p} = \frac{1}{\frac{1}{p} + \frac{\log_2 p}{N}}$

Calcular el mínimo

Estrategia #2

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados en paralelo con un árbol binario.

Modificar el código introdujo un *overhead* :

$$T_0 = pT_p - T1 = p\left(\frac{N}{p} + \log_2 p\right) - N = p\log_2 p$$

¿Cuándo es esto comparable al costo secuencial? – *cost optimal*

Calcular el mínimo

Estrategia #2

- Tomamos el arreglo de largo N y lo dividimos entre p procesadores.
- Cada procesador tiene un arreglo de largo N/p.
- Cada procesador busca un mínimo (local) en un total de N/p operaciones.
- Buscamos el mínimo (global) entre los p resultados en paralelo con un árbol binario.

El *Speedup* es $S_p = \dfrac{T_1}{T_p} = \dfrac{N}{\frac{N}{p} + \log_2 p} = \dfrac{1}{\frac{1}{p} + \frac{\log_2 p}{N}}$

La eficiencia es $E_p = \dfrac{1}{1 + \frac{p \log_2 p}{N}}$: si $p \log_2 p \sim N$, entonces $E_p \sim \mathrm{const}$

Calcular el mínimo

Estrategia #2

Escalabilidad fuerte (N fijo):

$$E_p = \frac{1}{1 + \frac{p \log_2 p}{N}} \xrightarrow{p \to \infty} 0.$$

Mejor que la Estrategia #1: el overhead crece como $p \log_2 p$ en lugar de p^2.

Escalabilidad débil ($M = N/p$ fijo, $N = pM$):

$$T_p = M + \log_2 p, \qquad E_p^w = \frac{M}{M + \log_2 p} \xrightarrow{p \to \infty} 0.$$

El overhead crece logarítmicamente: para M grande, $E_p^w \approx 1$.

Calcular el mínimo

Estrategia #3

- Tomamos el arreglo de largo N y generamos el arbol binario para calcular el mínimo.
- El árbol tiene largo $\log_2 N$.
- Cada nivel del árbol se puede calcular en paralelo: $\frac{N}{2^k}$ operaciones.
- Cada procesador realiza $\frac{N}{p2^k}$ operaciones en cada nivel.

Calcular el mínimo

Estrategia #3

- Tomamos el arreglo de largo N y generamos el arbol binario para calcular el mínimo.
- El árbol tiene largo $\log_2 N$.
- Cada nivel del árbol se puede calcular en paralelo: $\frac{N}{2^k}$ operaciones.
- Cada procesador realiza $\frac{N}{p2^k}$ operaciones en cada nivel.

El tiempo con un procesador es $T_1 = N$,

y con p procesadores es $T_p = \sum_{k=1}^{[\log_2 N]} \left[\frac{N}{p2^k}\right] \leq \frac{N}{p} \log_2 N$ (pesimista).

Calcular el mínimo

Estrategia #3

- Tomamos el arreglo de largo N y generamos el arbol binario para calcular el mínimo.
- El árbol tiene largo $\log_2 N$.
- Cada nivel del árbol se puede calcular en paralelo: $\frac{N}{2^k}$ operaciones.
- Cada procesador realiza $\frac{N}{p2^k}$ operaciones en cada nivel.

El *Speedup* es $S_p = \dfrac{T_1}{T_p} \approx \dfrac{N}{\frac{N}{p}\log_2 N} = \dfrac{p}{\log_2 N}$

La eficiencia es $E_p = \dfrac{1}{\log_2 N}$: $E_p \sim \mathrm{const}$ pero muy bajo.

Calcular el mínimo

Estrategia #3

Escalabilidad fuerte (N fijo):

$$E_p \approx \frac{1}{\log_2 N} \sim \text{const.}$$

La eficiencia no cae con p, pero es inherentemente baja: está acotada por el tamaño del problema.

Escalabilidad débil ($M = N/p$ fijo, $N = pM$):

$$T_p = M \log_2(pM), \qquad E_p^w = \frac{1}{\log_2(pM)} \xrightarrow{p\to\infty} 0.$$

Decrece logarítmicamente: más lento que #1 y #2, pero sin eficiencia constante.

Conclusión: ¿qué estrategia es mejor?

En escalabilidad débil, la Estrategia #2 domina: su overhead crece solo como $\log_2 p$, por lo que con M grande, $E_p^w \approx 1$.

¿Cuándo podemos tener M grande?

- **El problema crece naturalmente:** más datos, mayor resolución. Agregamos procesadores para mantener el tiempo constante.
- **Tenemos control sobre M:** podemos elegir cuánto trabajo hace cada procesador.
- **El problema no cabe en un nodo:** forzamos escalabilidad débil porque no hay otra opción.

Método de Monte Carlo

La idea central: usar muestreo aleatorio para estimar un valor esperado difícil de calcular directamente:

$$\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{i=1}^{N} f(x_i), \qquad x_i \sim p(x).$$

- f es el **observable**: la cantidad que queremos medir.
- Se generan N muestras x_i y se evalúa f en cada una.
- Por la ley de grandes números, el promedio converge al valor real. El error cae como $1/\sqrt{N}$.
- Aplicaciones: integración numérica, simulación, aprendizaje automático, finanzas.

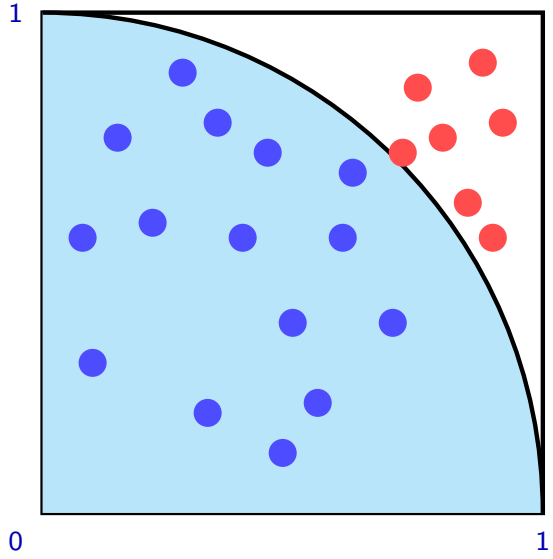
Clave para paralelismo: cada muestra es independiente. No hay comunicación durante el cómputo, solo una reducción al final.

Monte Carlo: estimación de π

- Generamos N puntos (x, y) con $x, y \sim U(0, 1)$.
- Un punto cae dentro del círculo si $x^2 + y^2 \leq 1$.
- La probabilidad de caer dentro es $\frac{\pi}{4}$.
- Estimamos:

$$\pi \approx 4 \cdot \frac{\text{puntos dentro}}{N}.$$

- El error cae como $1/\sqrt{N}$.



Monte Carlo: paralelismo

Cada procesador genera $M = N/p$ puntos de forma independiente. Al final, una reducción combina los conteos ($\log_2 p$ pasos).

$$T_1 = N, \qquad T_p = \frac{N}{p} + \log_2 p$$

$$E_p^w = \frac{M}{M + \log_2 p}$$

M es libre: para M grande, $E_p^w \approx 1$.

Monte Carlo: ejemplo concreto

Supongamos $N = 10^9$ puntos en total, distribuidos en $p = 1000$ procesadores:

$$M = \frac{N}{p} = 10^6 \text{ puntos por procesador.}$$

Eficiencia débil actual:

$$E^w_{1000} = \frac{10^6}{10^6 + \log_2 1000} \approx \frac{10^6}{10^6 + 10} \approx \boxed{0.99999}$$

¿A partir de qué p baja del $50\,\%$?

$$\frac{M}{M + \log_2 p} < 0{,}5 \implies \log_2 p > M = 10^6 \implies p > 2^{10^6} \approx 10^{300\,000}.$$